

项目面试问答（推荐算法岗位，公式加强版）

说明： 下面的问题设计遵循“重点追问第一个项目，次重点追问第二个项目，第三个项目作为科研与建模能力补充”的思路。相比上一版，这一版把关键算法公式、训练目标和项目改造点都补全了，便于你在面试时既能讲业务动机，也能讲清楚技术细节。

一、重点项目：基于元学习的冷启动 ID 嵌入生成推荐框架

Q1. 你这个项目的核心问题是什么？为什么冷启动 item 会影响推荐效果？

参考回答： 这个项目解决的是推荐系统中的 **item 冷启动** 问题。在 CTR 预估里，模型通常写成“Embedding + MLP”的形式。给定用户特征、item 特征和上下文特征后，模型输出点击概率

$$\hat{y} = f_{\Theta}(x),$$

其中 Θ 包含主网络参数和各类 embedding 参数。训练时一般采用二元交叉熵损失：

$$L_{CTR} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right].$$

问题在于：对一个新 item 而言，它的 **item_id** 是独有特征，只能依赖自身有限交互来更新；而像类目、品牌、标题等属性特征可以从大量相似 item 共享统计规律。所以冷启动的本质不是“模型不会学”，而是 **冷启动 item 的 ID embedding 学不好**，从而拖累整体排序效果。

Q2. 为什么你的方案是“生成一个好的初始 ID embedding”，而不是完全不用 ID 特征？

参考回答： 因为在工业推荐里，ID embedding 和内容特征不是替代关系，而是互补关系。内容特征负责泛化，ID 特征负责承载更细粒度的协同过滤信息。如果完全去掉 ID，只靠内容特征，模型会丢失很多行为层面的区分能力；如果只依赖 ID，又会在冷启动阶段因数据稀缺而失效。所以我的做法是学习一个生成器，为冷启动 item 产生一个更好的初始表示 r_i ，再用它替换原本随机初始化的 item ID embedding，送入已经训练好的推荐主网络。

Q3. 你这里为什么能用元学习？task 是怎么定义的？

参考回答： 元学习的关键是“以任务为单位训练，而不是以单条样本为单位训练”。在经典 MAML 中，设任务集合为 $\{\mathcal{T}_i\}$ ，每个任务包含支撑集（support set）和查询集（query set）。标准元目标可以写成

$$\min_{\phi} \sum_i L_{\mathcal{T}_i}^{\text{query}}(\theta'_i),$$

其中内层更新为

$$\theta'_i = \phi - \alpha \nabla_{\phi} L_{\mathcal{T}_i}^{\text{support}}(\phi).$$

这里的含义是：学习一个好的初始化参数 ϕ ，使得模型在新任务上经过极少量梯度更新后就能快速适配。

在我的项目里，我把“学习一个 item 的冷启动 ID embedding”看作一个任务。也就是说，每个老 item 都可以构造成一个 task，通过许多老 item 的任务训练，生成器学到一种

跨 item 共享的能力：如何根据属性特征和少量交互，为新 item 快速给出一个适合推荐任务的初始嵌入。

Q4. 你这个项目和经典 MAML 的关系是什么？有什么改造？

参考回答：经典 MAML 更强调

$$\theta'_i = \phi - \alpha \nabla_{\phi} L_i^{\text{support}}(\phi), \quad \phi \leftarrow \phi - \beta \sum_i \nabla_{\phi} L_i^{\text{query}}(\theta'_i).$$

也就是说，标准 MAML 的外层目标主要看 query loss。

而我这个项目借鉴了“内层适配 + 外层更新”的思想，但把它改造成了更贴合冷启动推荐的两阶段优化。具体来说，我把冷启动过程拆成：

- 完全冷启动阶段：0 或极少交互时，初始 embedding 能否直接支持预测；
- 预热阶段：少量交互到来后，这个 embedding 再更新一步，能否继续支持预测。

因此我的总目标不是单独的 query loss，而是

$$L = w_1 L_{\text{cold}} + w_2 L_{\text{warm}} + w_3 L_{\text{aux}},$$

其中前两项分别对应完全冷启动和预热阶段的预测损失，第三项是对比学习辅助损失。

Q5. 你先讲一下生成器是怎么构建的。

参考回答：生成器并不是只看当前冷启动 item 自己的属性，而是同时引入了“和它属性相似的老 item 群组信息”。

先给当前冷启动 item 找到最相似的 K 个老 item，把这些老 item 的 ID embedding 做平均池化，得到群组表示：

$$f_{v_i}^0 = \text{mean-pooling}(V),$$

其中 V 表示这 K 个相似老 item 的 embedding 集合。

然后把群组特征和 item 自身属性特征拼接起来：

$$z_i = f_{v_i}^0 \parallel f_{v_i}^1 \parallel \dots \parallel f_{v_i}^n,$$

其中 $\parallel$ 表示向量拼接， $f_{v_i}^1, \dots, f_{v_i}^n$ 表示 item 自身各属性特征的 embedding。

再把 z_i 输入一个 MLP 生成器，输出冷启动 item 的初始 ID embedding：

$$r_i = \gamma \tanh(W z_i).$$

其中 W 是生成器参数， γ 是缩放系数。面试时我会强调：这个生成器本质上是在学一个“属性空间 $\rightarrow$ ID 表示空间”的映射，但它不是静态回归，而是通过元学习训练成“易于少样本适配”的初始化器。

Q6. 你怎么给冷启动 item 找最相似的 K 个老 item？为什么不用 ID embedding 相似度？

参考回答：不用 ID embedding 相似度，是因为新 item 的 ID embedding 还没训好，甚至接近随机初始化，这时候拿它去做相似检索不可靠。

所以我用的是 **属性相似**。做法比较朴素但稳定：对每个老 item 和当前冷启动 item，比对它们的属性值是否相同，相同一个属性就加一分，最后按得分排序，取前 K 个。这种做法虽然简单，但优点是可解释、鲁棒，而且在冷启动阶段比基于未训练好的 ID 表示更可靠。

Q7. 你这里的对比学习具体怎么做？为什么它能帮助冷启动？

参考回答： 对比学习部分的目标，是让生成器输出的 embedding 在表示空间里满足“同源相近、异源相远”。

对于每个冷启动 item v_i ，先把它的属性特征集合拆成两个子集 F_i^a 和 F_i^b ，分别送入同一个生成器，得到两个 embedding：

$$r_i^a = \text{generator}(F_i^a), \quad r_i^b = \text{generator}(F_i^b).$$

它们来自同一个 item，因此是正样本对；而来自其他 item 的 embedding 则作为负样本。相似度用余弦相似度衡量：

$$s(r_i^b, r_j^a) = \frac{(r_i^b)^\top r_j^a}{\|r_i^b\| \|r_j^a\|}.$$

辅助对比损失写成

$$L_{\text{aux}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s(r_i^a, r_i^b)/\tau)}{\sum_{j=1}^N \exp(s(r_i^b, r_j^a)/\tau)},$$

其中 τ 是温度系数。

这个损失的作用是：让同一个 item 经过不同特征子集生成的 embedding 更接近，让不同 item 的 embedding 更可分，从而提高冷启动表示空间的结构性和判别性。

Q8. 属性子集为什么要专门设计划分方式？随机切分不行吗？

参考回答： 随机切分不一定不行，但效果未必稳定。因为如果切出来的两个子集高度相关，那它们产生的两个 embedding 天然就很像，对比学习任务会变得过于简单，起不到真正的表示约束作用。

所以我的做法是：先随机选一个种子特征 f_{seed} ，再根据余弦相似度找到和它最相关的一半特征，把它们放进同一个子集，剩余特征放到另一个子集。这样能增大两个子集的信息差异，让对比任务更有效。

Q9. 你说做了“嵌入去噪”，具体公式是什么？

参考回答： 冷启动 item 的样本少，异常点击、误触点击、离群用户都会对它的 ID embedding 造成很大扰动。所以我用冷启动 item 已有交互用户的 embedding 做平均池化，构造一个去噪因子：

$$d_i = \text{mean-pooling}(e_{u_1}, e_{u_2}, \dots, e_{u_n}),$$

其中 e_{u_k} 表示和该冷启动 item 发生过交互的第 k 个用户的 ID embedding。

然后把生成器输出与去噪因子相加，得到最终初始表示：

$$r_i^{\text{en}} = r_i + d_i.$$

平均池化的好处是能减弱离群样本的影响，因此面试时我会把它解释为：用有限但相对稳定的用户侧统计信息，对冷启动 item 的初始表示做一次鲁棒校正。

Q10. 你项目里的优化目标是什么？请把公式完整讲出来。

参考回答： 这个问题其实就是在问损失函数设计。

首先，在完全冷启动阶段，用生成器产生的 embedding r_i^{en} 替换 item 原始随机 ID embedding，在第一个 batch 上做预测，得到冷启动损失：

$$L_{\text{cold}} = \frac{1}{M} \sum_{j=1}^M \left[-y_{aj} \log \hat{y}_{aj} - (1 - y_{aj}) \log(1 - \hat{y}_{aj}) \right].$$

然后，为了模拟 item 在拿到少量数据后进入预热阶段，让该 embedding 在冷启动损失上做一步梯度更新：

$$r'_i = r_i^{\text{en}} - \eta \frac{\partial L_{\text{cold}}}{\partial r_i^{\text{en}}},$$

这里 η 是步长。

接着，用更新后的 r'_i 替换 embedding，在第二个 batch 上做预测，得到预热损失：

$$L_{\text{warm}} = \frac{1}{M} \sum_{j=1}^M \left[-y_{bj} \log \hat{y}_{bj} - (1 - y_{bj}) \log(1 - \hat{y}_{bj}) \right].$$

最后，把冷启损失、预热损失和对比辅助损失结合起来：

$$L = w_1 L_{\text{cold}} + w_2 L_{\text{warm}} + w_3 L_{\text{aux}}.$$

所以这个项目的核心思想，不是只追求“0 样本时有个好 embedding”，而是同时要求它在“少量样本更新之后依然工作得好”。

Q11. 你为什么要把训练拆成两阶段：先训主网络，再冻结主网络去训生成器？

参考回答：因为如果主网络和生成器一起从头训练，优化目标会很不稳定。主网络本身还没学好时，生成器输出的 embedding 好坏很难判断，二者会相互干扰。

所以我先用老 item 的充足数据把推荐主网络训好，得到稳定的特征抽取和点击预测能力；然后冻结主网络参数，只训练冷启动框架。这样生成器面对的是一个固定的下游任务，优化目标会更清晰，也更接近“为一个已存在的工业排序模型生成更好的冷启动初始化”。

Q12. 你为什么用老 item 的数据来训练冷启动框架？这听起来不矛盾吗？

参考回答：不矛盾，反而这是关键。因为真正的新 item 没有足够交互，根本无法稳定地产生 L_{cold} 和 L_{warm} 。所以训练时只能用数据充足的老 item 去模拟冷启动 item 的一生：先假装它刚上线，只有极少样本，计算冷启动损失；再假装它又积累了一点样本，计算预热损失。本质上，Old2 不是“真正的冷启动集”，而是“用于模拟冷启动过程的元训练任务来源”。

Q13. 相比普通梯度下降或者普通属性映射模型，你的方法优势到底在哪里？

参考回答：我认为优势有三层。

第一层是 **初始化更好**。普通方法往往是随机初始化或者用一个静态 MLP 回归 embedding，而我这里用元学习显式优化“初始化是否适合少样本快速适配”。

第二层是 **冷启和预热统一优化**。不是只看 0 样本阶段，也不是只看少量样本阶段，而是同时兼顾两者。

第三层是 **表示空间更稳**。通过对比学习约束表示结构，通过用户交互平均池化做去噪，使生成的 embedding 不只是“能用”，而且更鲁棒。

Q14. 你这个项目如何评估效果？为什么用 GAUC？

参考回答： 我主要看的是冷启动 item 上的离线排序效果，比如 AUC 和 GAUC。GAUC 更适合推荐场景，因为它会在分组层面衡量排序效果，比单纯整体 AUC 更能反映模型在不同用户或不同 item 子群上的稳定性。

评估方式也很直接：

- 用随机初始化的冷启动 item ID embedding，得到一个 GAUC；
- 用生成器产生的冷启动 item ID embedding，替换后再算一个 GAUC。

如果

$$\text{GAUC}_{\text{generator}} > \text{GAUC}_{\text{random}},$$

就说明冷启动框架有效。你文档里的表述是离线 GAUC 从 0.85 提升到 0.86，核心结论就是：生成器确实能提升冷启动 item 的排序质量。

二、次重点项目：基于风险估计的多智能体强化学习集成评论家网络（ECREAT）

Q1. 你这个项目解决的核心问题是什么？

参考回答： 这个项目解决的是多智能体强化学习中的 风险感知价值估计与安全探索 问题。在随机博弈框架下，智能体 i 的目标通常写成

$$J^i = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t^i(s_t, a_t) \right].$$

如果采用 actor-critic 或策略梯度方法，策略梯度一般可以写为

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E} [\nabla_{\theta_i} \log \pi_i(a_i | o_i) q_i(x, a_1, \dots, a_N)].$$

问题在于：在多智能体环境中，不确定性和对手策略会让 Q 值估计更不稳定，而高风险区域的探索又可能带来严重代价。所以这个项目的核心不是单纯提高均值回报，而是让评论家网络能显式感知风险，并据此调整训练目标。

Q2. 为什么这里要引入 VaR 和 CVaR？请把定义写出来。

参考回答： VaR 和 CVaR 是经典风险度量。设损失随机变量为 L ，置信水平为 α ，则

$$\text{VaR}_{\alpha} = \inf \{ l \in \mathbb{R} : \mathbb{P}(L \leq l) \geq \alpha \}.$$

它刻画的是在给定置信水平下的分位损失阈值。

CVaR 则进一步关注超出该阈值后的平均尾部损失：

$$\text{CVaR}_{\alpha} = \mathbb{E}[L | L > \text{VaR}_{\alpha}].$$

相比只看均值回报，CVaR 更关注最坏那部分结果的平均表现，所以特别适合安全约束、高风险惩罚显著的场景。

在 ECREAT 里，作者把集成 Q 网络输出看成一个价值分布，从而把风险度量引入到状态-动作对层面。

Q3. ECREAT 里怎么从集成 Q 网络构造风险估计？

参考回答： 设集成 Q 网络由多个子模型组成：

$$Q = \{q_1, q_2, \dots, q_K\}.$$

对于某个状态-动作对 (s, a) ，不同子模型会给出不同 Q 值预测，从而形成一个经验分布。在该分布上，文档把状态-动作风险写成

$$\text{VaR}_\alpha(s, a) = -\inf\{q \mid \phi_F(q \mid s, a) \geq \alpha\},$$

进一步得到

$$\text{CVaR}(s, a) = \frac{1}{1-\alpha} \int_\alpha^1 \text{VaR}_u(s, a) du.$$

然后再定义下一步的平均风险估计：

$$c_i = \mathbb{E}_{p, \pi}[\text{CVaR}(s_{t+1}^i, a_{t+1}^i) \mid s_t^i, a_t^i].$$

这一步的意义是：不只看当前动作值高不高，还看它会不会把智能体带到一个未来尾部风险更大的区域。

Q4. 你说是“集成评论家网络”，它的集成形式是什么？

参考回答：对于第 k 组智能体，其集成评论家的加权输出可以写成

$$q^k(s_i, a_i) = \sum_{j=1}^{n_k} \omega_j q_j^k(s_i, a_i; \theta_j^k),$$

其中 n_k 是该组内的子评论家数量， ω_j 是加权系数。

面试时我会强调两个点：第一，集成不是为了炫技，而是为了利用多个子模型输出的分歧来刻画不确定性；第二，这些子模型本身还能提供一种自举式的 Q 值集合，从而提高样本利用率和训练稳定性。

Q5. ECREAT 里的“歧义性”是什么？为什么能增强探索？

参考回答：对每个子评论家，定义它相对集成输出的偏离程度：

$$\nu_j(s, a) = (q_j(s, a) - q^k(s_i, a_i))^2.$$

然后定义集成的平均歧义性：

$$\bar{\nu}^k(s_i, a_i) = \sum_{j=1}^{n_k} \omega_j \nu_j^k(s_i, a_i) = \sum_{j=1}^{n_k} \omega_j (q_j^k(s_i, a_i) - q^k(s_i, a_i))^2.$$

如果子模型之间分歧很大，说明这个状态-动作对的不确定性更高，可以把它视作一种“值得探索”的信号。这和 UCB 的思想是一致的，即在价值之外再加一个不确定性奖励项，鼓励模型探索尚未充分学习清楚的区域。

Q6. ECREAT 最关键的训练目标是什么？请你把完整公式讲出来。

参考回答：ECREAT 把探索增强项和风险惩罚项同时纳入 TD 目标。文档中的修正目标写为

$$\tilde{y}_t^k = y_t^k - \lambda_1 \bar{\nu}^k - \lambda_2 c_i = r_{t+1}^i + \gamma q^k(s_{t+1}, a_{t+1}; \theta^{k+}) - \lambda_1 \bar{\nu}^k - \lambda_2 c_i.$$

其中：

- λ_1 是探索系数，控制歧义性项；
- λ_2 是风险系数，控制未来 CVaR 惩罚；
- θ^{k+} 是目标网络参数。

最终的 Q 网络损失为

$$L(\theta^k) = \mathbb{E}_{p,\pi} \left[(q^k(s_t^i, a_t^i) - \tilde{y}_t^k)^2 \right].$$

这个式子是整个方法的核心。因为它同时体现了三个东西：

- (a) 普通强化学习的 TD 学习；
- (b) 通过歧义性促进探索；
- (c) 通过 CVaR 对未来高风险状态进行惩罚。

Q7. 如果面试官问：这个方法相比 MADDPG 到底多了什么？

参考回答： 可以概括成三点。

第一，多了**集成评论家**，不再依赖单一 critic 的点估计。

第二，多了**风险度量**，不只看期望回报，还把尾部风险显式纳入目标。

第三，多了**歧义性驱动的探索机制**，不是简单靠 ε -greedy 或动作噪声，而是用评论家之间的分歧来引导探索。

所以它不是简单在 MADDPG 上“加个正则项”，而是把**价值估计、风险控制、探索增强**三件事统一进了一个训练目标里。

三、补充项目：多智能体卫星对抗仿真环境设计与实现

Q1. 这个环境的动力学基础是什么？为什么选 CWH/Hill 方程？

参考回答： 这个环境基于 Clohessy-Wiltshire-Hill 相对轨道动力学方程，用于描述目标卫星附近的相对运动。如果把相对位置写成 (x, y, z) ，控制输入写成 (u_x, u_y, u_z) ，标准线性化形式可以写成

$$\ddot{x} - 2n\dot{y} - 3n^2x = u_x,$$

$$\ddot{y} + 2n\dot{x} = u_y,$$

$$\ddot{z} + n^2z = u_z,$$

其中 $n = \sqrt{\mu/a^3}$ 是参考圆轨道的角速度。

之所以选 CWH/Hill，是因为它能在目标卫星附近给出一个较简洁、可控、适合强化学习迭代训练的线性动力学近似，既保留轨道相对运动的主要耦合项，又不至于让环境仿真过于昂贵。

Q2. 你这个环境里的“太阳致盲机制”怎么建模？

参考回答： 太阳方向向量记为 $\mathbf{s}_t$ ，在 CWH 坐标系下按轨道角速度反向旋转：

$$\mathbf{s}_{t+1} = R_z(-n\Delta t) \mathbf{s}_t,$$

其中

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

对于观察者 i 看向目标 j ，先定义单位视线方向

$$\mathbf{v}_{ij} = \frac{\mathbf{p}_j - \mathbf{p}_i}{\|\mathbf{p}_j - \mathbf{p}_i\|}.$$

若

$$\mathbf{v}_{ij} \cdot \mathbf{s}_t > \cos(\theta_{\text{blind}}),$$

则认为观察者 i 被太阳致盲，无法有效探测目标 j 。

进一步，探测条件写成

$$\text{can_detect}[i, j] = \text{in_range}[i, j] \wedge \neg \text{is_blinded}[i, j].$$

所以这个机制本质上是在观测模型里引入了一个和太阳方位相关的几何可见性约束。

Q3. 你为什么选择 JAX 来做并行化支撑？

参考回答： 因为这类仿真环境通常需要大量并行 rollout 才能支撑强化学习训练，JAX 在向量化计算、自动微分和并行执行方面都有优势，比较适合大规模环境批量仿真。所以选择 JAX，主要是出于训练效率和环境扩展性的考虑。

Q4. 如果面试官问你在这个项目里最能体现工程能力的点是什么？

参考回答： 我会强调两点。第一是环境机制设计，包括动力学建模、任务奖励和观测空间定义；第二是可视化系统实现，把多智能体轨迹、探测范围和攻击连线动态展示出来，这样不仅方便调试，也能直接支持策略效果分析和结果汇报。

Q5. 如果面试官问你，这个项目最能体现什么能力，你怎么答？

参考回答： 我会说这个项目最能体现三类能力。

第一是 **建模能力**：把轨道相对运动、太阳致盲、攻击判定和 HVT 保护目标统一到一个可交互环境里。

第二是 **奖励设计能力**：能够把“进攻、探测、防守、避险”这些高层任务目标拆成可学习的回合级奖励项。

第三是 **工程落地能力**：环境基于 JAX 实现，支持并行 rollout 和后续 MARL 算法训练，同时还做了三维可视化，方便调试与结果分析。

四、收尾总结：推荐岗位里怎么把这三个项目串起来

如果面试官最后问：这三个项目怎么体现你适合推荐算法岗位？可以这样回答：

这三个项目虽然业务背景不同，但核心能力是相通的。

第一个项目体现的是我围绕推荐场景中的冷启动问题，能够把业务痛点抽象成可优化的模型目标，并且能把元学习、对比学习和去噪机制结合起来设计完整方案。

第二个项目体现的是我对不确定性建模和训练稳定性的理解，这在推荐系统里同样重要，因为推荐模型也会面对样本偏置、长尾分布和高风险曝光问题。

第三个项目体现的是我的环境建模和工程实现能力，说明我不只是会调包训练模型，也能从状态、动作、奖励和约束出发搭建一个完整可验证的算法平台。

如果映射到推荐算法岗位，我认为我最匹配的地方是：我不仅能做点击率预估、冷启动、表示学习这类核心建模问题，也比较重视训练目标设计、实验评估和方法落地的完整闭环。